Reviewer Pack — Minimal Rank of Geometric Loci vs Resolution Proof Length fo…

Entry e84d0fa2c956 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 02:06:55 UTC

Minimal Rank of Geometric Loci vs Resolution Proof Length for Tseitin Formulas

Entry ID: `e84d0fa2c956` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 02:06:55 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Tseitin Formulas

Statement:

> {'content': ['For a given Tseitin formula F, let L(G) denote the geometric locus of vertices in the associated graph G that have at least one loop.', 'Then, the resolution proof length for F is upper-bounded by 2^(O(|L(G)|))', 'where |L(G)| denotes the size of the geometric locus.'], 'invariant': 'The size of the geometric locus of vertices with loops in the graph associated with a Tseitin formula'}

Rationale (proposer's reasoning):

> {'content': ['Geometric group theory has been used to study structures in geometry that are difficult to describe explicitly. The geometric locus, which captures the set of points sharing a common property, might reveal inherent complexity in the structure.', 'By linking this concept to Tseitin formulas, we may uncover new insights into the complexity of resolution proofs.', 'The proposal suggests that the presence of loops in the graph could lead to an exponential increase in proof length, which is a characteristic of NP-complete problems.']}

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `82f46e4b5d06c70f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all 30 seeds, the mean resolution proof length is within an order of magnitude (2^(O(|L(G)|))) of the expected bound and no seed produces a resolution proof length that exceeds this bound by more than 50%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `geometric group theory AND tseitin formulas resolution proof complexity` - `resolution proof length tseitin formulas geometric locus vertices` - `upper bound 2^(O(|L(G)|)) geometric group theory tseitin formulas`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import random
import math

def generate_graph(clauses):
    n = len(clauses)
    graph = [[] for _ in range(n)]
    for clause in clauses:
        if len(clause) >= 3 and clause[-2] != '-':
            v = int(clause[-2]) - 1
            u = int(clause[-3]) - 1
            graph[u].append(v)
            graph[v].append(u)
    return graph

def find_loops(graph):
    n = len(graph)
    visited = [False] * n
    parent = [-1] * n

    def dfs(node, p):
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                parent[neighbor] = node
                if dfs(neighbor, node):
                    return True
            elif neighbor != p:
                return True
        return False
```

```
    for i in range(n):
        if not visited[i]:
            if dfs(i, -1):
                return True
    return False

def resolution_proof_length(clauses):
    n = len(clauses)
    stack = []
    literals = set()

    def resolve(lit1, lit2):
        nonlocal literals
        if abs(lit1) == abs(lit2):
            literals.remove(abs(lit1))
            literals.discard(-abs(lit1))
            return True
        return False

    for clause in clauses:
        if len(clause) > 0 and clause[0] != '-':
            literals.update([int(x) for x in clause])

    while literals:
        lit = next(iter(literals))
        stack.append(lit)
        literals.remove(abs(lit))

        while stack:
            top_lit = stack[-1]
            found_resolvent = False
            for clause in clauses:
                if len(clause) > 0 and clause[0] != '-':
                    if -top_lit in clause:
                        resolvent = [x for x in clause if x != -top_lit]
                        if resolve(top_lit, -resolvent[0]):
                            found_resolvent = True
                            break
            if not found_resolvent:
                stack.pop()
                literals.add(abs(top_lit))

    return len(stack)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    clauses = []
    for _ in range(n):
        num_vars = random.randint(2, n)
        clause = ['-' + str(random.randint(1, num_vars))]
        while len(clause) < 3 or (len(clause) == 3 and clause[-2] == '-'):
            clause.append(str(random.randint(1, num_vars)))
        clauses.append(clause)

    graph = generate_graph(clauses)
```

```
    has_loops = find_loops(graph)
    proof_length = resolution_proof_length(clauses)

    return {
        "metric_name": "Resolution Proof Length",
        "metric_value": proof_length,
        "instances_tested": 1,
        "conjecture_holds": proof_length <= 2 ** (math.ceil(math.log2(len([v for v in graph if has
        "counterexample": "" if has_loops else "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14692
2	preregistration	ollama_remote	glm4:latest	0	0	10079
3	novelty	ollama_remote	glm4:latest	0	0	8346
4	novelty	ollama_remote	glm4:latest	0	0	8941
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14616
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13907
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14277
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12855
9	judge	ollama_remote	glm4:latest	0	0	11532

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 109245 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e84d0fa2c956.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e84d0fa2c956.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e84d0fa2c956.tar.gz` (if generated)